OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

9: I/O streams, Communication and Events

Jan H. Mikkelsen

Cyber og Computerteknologi

2. semester – KBH – F26

AGENDA

- Input and outputs
- Data streams
  - Byte streams
  - Character streams
  - Buffered streams
  - Data streams
  - Object serialization
  - I/O from keyboard
- Network I/O
  - UDP sockets
  - TCP sockets
- Events
  - Graphical types of events
  - Network types of events
- The important stuff
- Assignments

- To be able to understand streams in java
- To be able to use different classes that relies on streams, file, networking etc.
- To understand concept of events and event driven java

KEYBOARD INPUT AND SCREEN OUTPUT

- We have several ways to read input from the keyboard
  - The Scanner class models a tool to obtain input from the keyboard

```
Scanner keyboard = new Scanner(System.in);
System.out.println("enter an integer");
int myint = keyboard.nextInt();
```

- However, the class is more sophisticated than that. The constructor is overloaded and therefore allows for several input sources
  - Scanner(File source); Scanner(InputStream source); Scanner (Path source)
- This enable us to treat the keyboard the same way as a file, a data transmission stream and more
  - System.in is an input stream from the System object specified by the host environment JVM input – default is the keyboard

PRINTING TO THE SCREEN

- This is nothing new and we have seen this line of code a lot of times

```
System.out.println(”Hello world!!!");
```

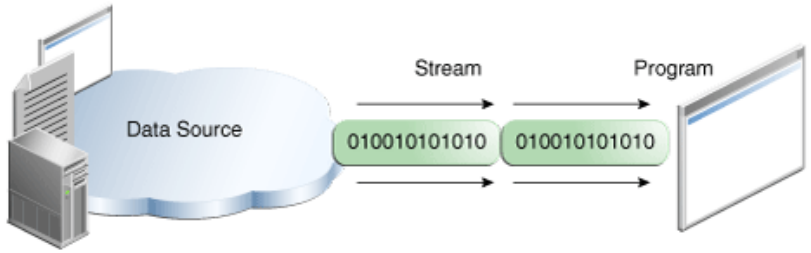
- System.out is actually an output stream, which is a class that has some methods like print() and println()
  - So by writing the above line, we actually get hold of an instance of an object and use a method – that is really OO
- The ***System*** class is based on a configuration specific to the host and allows interaction with the system
- But what are those *streams*???

AGENDA

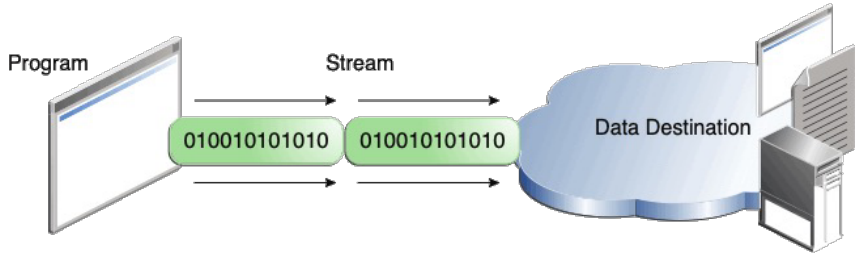
- Input and outputs
- Data streams
  - Byte streams
  - Character streams
  - Buffered streams
  - Data streams
  - Object serialization
  - I/O from keyboard
- Network I/O
  - UDP sockets
  - TCP sockets
- Events
  - Graphical types of events
  - Network types of events
- The important stuff
- Assignments

DATA STREAMS

- A stream in Java is a model of an input and/or output with a sequence of data in between



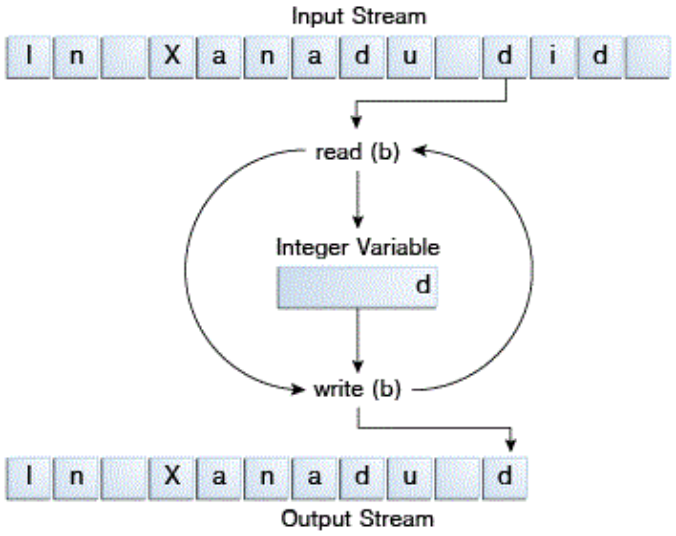
- Streams are twofold
  - Input streams relates to reading
  - Output streams relates to writing



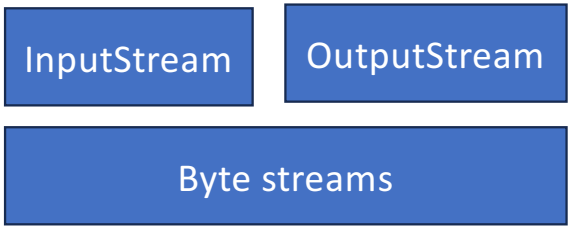
- These two models creates a foundation for all subsequent cases where data exchanges exists

BYTE STREAMS

- The simplest and lowest level type of streams is the byte stream
- Byte streams are modelled by abstract classes
  - InputStream
  - OutputStream
- From these input/output streams access to e.g. files or sockets can occur



https://docs.oracle.com/javase/tutorial/essential/io/bytestreams.html



FILE I/O WITH BYTE STREAMS

- FileInputStream – a class that allows reading from a file
- As any resource, we need to ”open” it before using it

```
try {
        FileInputStream fileInputStream = new FileInputStream("path/to/your/file");
        …  here we do the reading part
        fileInputStream.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
```

- Then, if successful we can proceed to fill some buffer

```
byte[] buffer = new byte[1024]; // Buffer to store the read bytes
int bytesRead; // Number of bytes read

while ((bytesRead = fileInputStream.read(buffer)) != -1) {
   … do stuff with the file content…
}
```

FILE I/O WITH BYTE STREAMS

- Maybe we need to do some conversion along the way
- Print the file content to the screen (no failure check – we assume ASCII characters)

```
// Example: Print the content of the file to the console
String content = new String(buffer, 0, bytesRead);
System.out.println(content);
```

- Notice that we here utilize a version of String class to work with the byte stream
- The code needed to convert content into printable characters is conveniently found in the String class
  - In fact, the String class offers a ton of string related functionality
  - Makes it easy to find relevant functionality - *I have a string I need to do something with, let me check the String class*

FILE I/O WITH BYTE STREAMS

- Conveniently enough, output is handled in same way

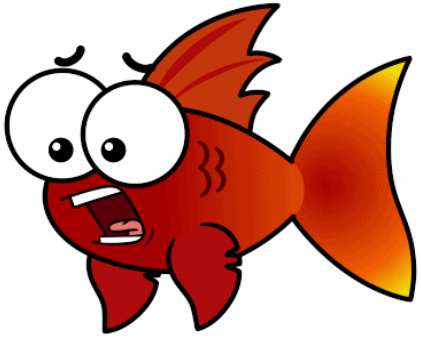
```
try {
        FileOutputStream fileOutputStream = new FileOutputStream("path/to/your/file");
        // Write bytes to the file using fileOutputStream
        fileOutputStream.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
```

- Then, once we have the outputStream, we need to convert our string into a sequence (array) of bytes and feed that into out outputStream

```
String data = "Hello, World!"; // Data to be written
byte[] bytes = data.getBytes(); // Convert the string to bytes

fileOutputStream.write(bytes);
```

BREAK – PAUSE(FISK)

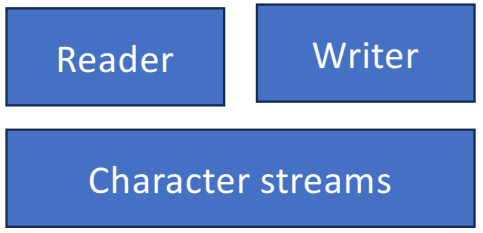


CHARACTER STREAMS

- Character streams are very similar to byte streams
- The difference is, that Java just works with characters instead of raw bytes

- Two classes are constructed on top of Character streams
  - Reader: for reading character streams
  - Writer: for writing character streams

- Subsequently, specialized classes inherits from these two classes



FILE I/O WITH CHARACTER STREAMS

- Two classes are based on Reader and Writer related to file I/O
  - FileReader
  - FileWriter
- The name of the readers may be confusing, as they relate to streams
  - Then again, we are working with streams through the FileReader/Writer classes
  - In both cases we use the read() and write() methods – this is a part of the OO design!!
- Here we copy a file char by char

```
FileReader inputStream = null;
FileWriter outputStream = null;


try {
  inputStream = new FileReader("xanadu.txt");
  outputStream = new FileWriter("characteroutput.txt");
  int c;
  while ((c = inputStream.read()) != -1) {
    outputStream.write(c);
  }
} finally {
  if (inputStream != null) {
    inputStream.close();
  }
  if (outputStream != null) {
    outputStream.close();
  }
}
```

FILE I/O WITH CHARACTER STREAMS

- In this example the stream variables are declared outside of try{} since they must be available to ‘finally’
- The ‘finally’ part always runs whether the copy succeeded or an exception was thrown
- Closing streams is critical and without this, file handles leak and the output file may not be fully flushed to disk
- Remembering the ‘finally’ bit is therefore very important

```
FileReader inputStream = null;
FileWriter outputStream = null;


try {
  inputStream = new FileReader("xanadu.txt");
  outputStream = new FileWriter("characteroutput.txt");
  int c;
  while ((c = inputStream.read()) != -1) {
    outputStream.write(c);
  }
} finally {
  if (inputStream != null) {
    inputStream.close();
  }
  if (outputStream != null) {
    outputStream.close();
  }
}
```

FILE I/O WITH CHARACTER STREAMS

- Since ‘finally’ MUST be included, modern (v7+) Java writes this more cleanly using try-with-resources, which handles closing automatically
- Same behaviour, but the ‘finally’ block and null checks are no longer needed as the compiler handles it

```
try (FileReader inputStream = new FileReader("xanadu.txt");
   FileWriter outputStream = new FileWriter("characteroutput.txt")) {
   int c;
   while ((c = inputStream.read()) != -1) {
      outputStream.write(c);
   }
}
```

```
FileReader inputStream = null;
FileWriter outputStream = null;


try {
   inputStream = new FileReader("xanadu.txt");
   outputStream = new FileWriter("characteroutput.txt");
   int c;
   while ((c = inputStream.read()) != -1) {
      outputStream.write(c);
   }
} finally {
   if (inputStream != null) {
      inputStream.close();
   }
   if (outputStream != null) {
      outputStream.close();
   }
}
```

OBSOLETSKI

BUFFERED STREAMS

- The prior examples work directly with the underlying OS file I/O
- In many cases, but not always, some intermediate buffer between the application and OS is useful, calling for buffered streams
- A buffer is added to a stream reader by ”wrapping” it into a buffer class, e.g.

```
inputStream = new BufferedReader(new FileReader("xanadu.txt"));
outputStream = new BufferedWriter(new FileWriter("characteroutput.txt"));
```

Character streams

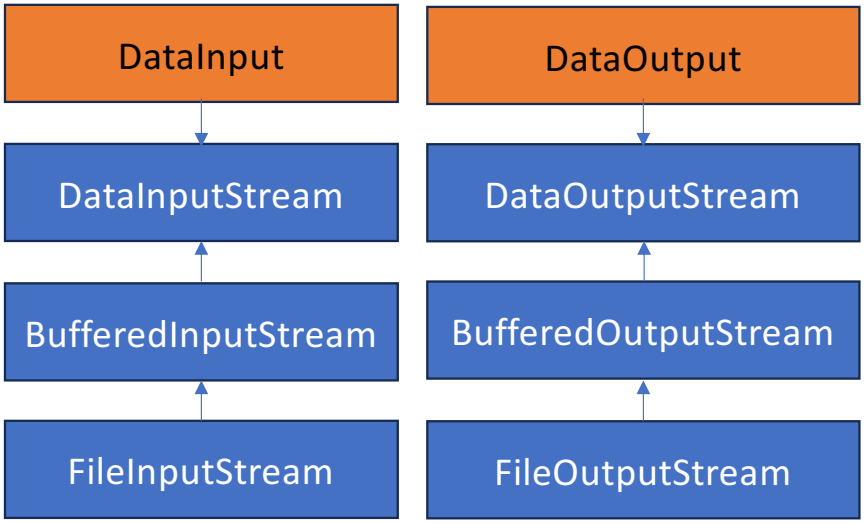
- The buffering classes are named differently whether it is character or byte streams – guess which is for what?

```
inputStream = new BufferedInputStream(new FileInputStream("xanadu.txt"));
outputStream = new BufferedOutputStream(new FileOutputStream("characteroutput.txt"));
```

Byte streams

DATA STREAMS

- Data streams support binary I/O of primitive data type values
- DataInputStream reads until it runs out of data and then it throws an EOFException
- Not returning -1 at EOF is unusual compared to most Java I/O
  - FileReader.read() returns -1




```
out = new DataOutputStream(new BufferedOutputStream(
        new FileOutputStream(dataFile)));
for (int i = 0; i < prices.length; i ++) {
   out.writeDouble(prices[i]);
   out.writeInt(units[i]);
   out.writeUTF(descs[i]);
}
```

```
in = new DataInputStream(new
        BufferedInputStream(new FileInputStream(dataFile)));
price = in.readDouble();
unit = in.readInt();
desc = in.readUTF();
System.out.format("You ordered %d" + " units of %s at $%.2f%n",
        unit, desc, price);
total += unit * price;
```

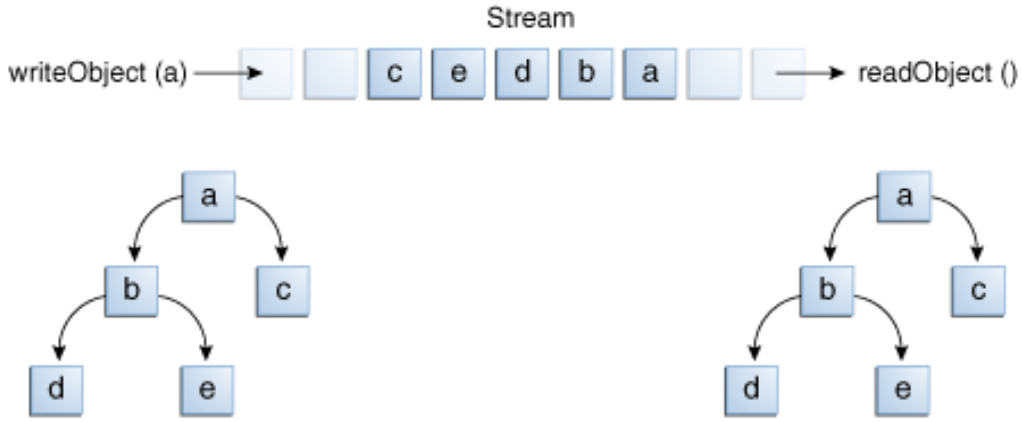
DATA STREAMS

- The standard pattern for handling this is as shown here
- Using an exception for normal control flow (EOF) is considered poor design
- Exceptions should signal unexpected conditions, not routine ones like reaching the end of a file
- This is a legacy design decision in Java and it has been criticised precisely because it mixes normal flow with error handling, making the code harder to read and reason about

```
try {
  while (true) {
    double d = in.readDouble(); // keeps reading
    System.out.println(d);
  }
} catch (EOFException e) {
  // end of file reached — this is normal, not an error
  System.out.println("Done reading");
} catch (IOException e) {
  // this however IS a real error
  System.out.println("Something went wrong: " + e.getMessage());
}
```

OBJECT SERIALIZATION

- Object stream classes are very useful when we wish to read/write complete objects
- However, the object is required to implement the interface Serializable
  - This is only a marker interface and does and asks nothing
- For each write object, there must be a read object as well
  - Simplification to save or exchange objects that may be complex to do so otherwise



https://docs.oracle.com/javase/tutorial/essential/io/objectstreams.html

```
ObjectOutputStream out;
Object ob = new Object();
out.writeObject(ob);
out.writeObject(ob);

ObjectInputStream in;
Object ob1 = in.readObject();
Object ob2 = in.readObject();
```

Object InputStream

Object OutputStream

Object streams

I/O TO/FROM THE COMMAND LINE

- It is VERY convenient to be able to read from the keyboard and write to the screen – if nothing else, then for testing
- Java offers three standard streams (most of which you know by now)
  - Standard input: System.in (byte stream for historical reasons)
  - Standard output: System.out (PrintStream based on character stream)
  - Standard error: System.err (PrintStream based on character stream)
- System.in is a byte stream, and this is a legacy from Java's early days when it mirrored C's stdin. In practice you almost never use it raw — you wrap it immediately
- In a very similar way, compared to the previous cases, the stream is wrapped around an input stream reader

```
InputStreamReader cin = new InputStreamReader(System.in);
```

I/O TO/FROM THE COMMAND LINE

- InputStreamerRead acts as a bridge converting the byte stream into a character stream, and BufferedReader adds the buffering. This is the same decorator pattern seen throughout Java I/O

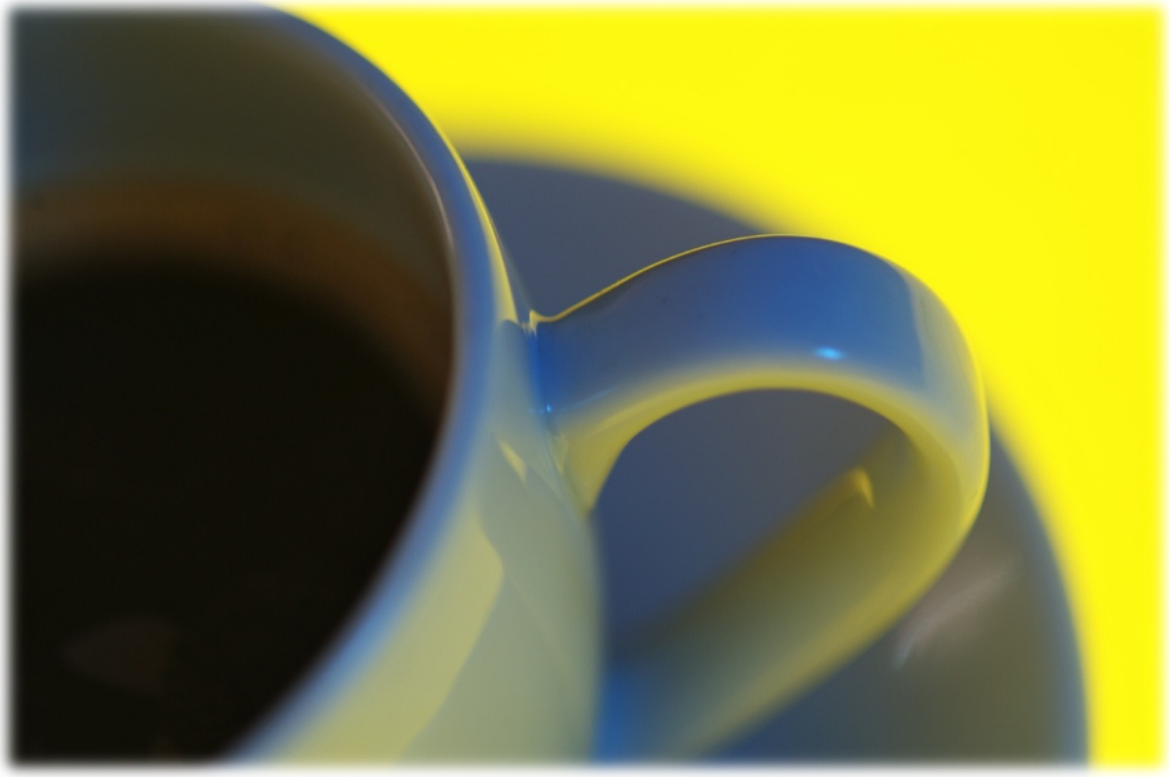
```
BufferedReader input = new BufferedReader(
new InputStreamReader(System.in));
```

```
Scanner sc = new Scanner(System.in);
```

- This is exactly what Scanner does, only here all the wrapping is internal to the method
- System.out versus System.err
  - Both are PrintStream but they serve different purposes
  - System.out is the normal program output and it can be redirected
  - System.err contain error messages and are typically shown in red in IDEs

```
//Redirect output to a file, errors still show on screen
java MyProgram > output.txt
```

”BIKER BREAK!”



FORMATTING OF STREAMS

- Stream classes that implement formatting
  - PrintStream for byte streams
  - PrintWriter for character streams
- For PrintStream the most useful is System.out
  - System.out.print ()
  - System.out.println()
  - System.out.format()
- Being a PrintStream, is what provides the convenience of println() as is handles type conversion internally. It accepts literally anything, which is why it is good for testing

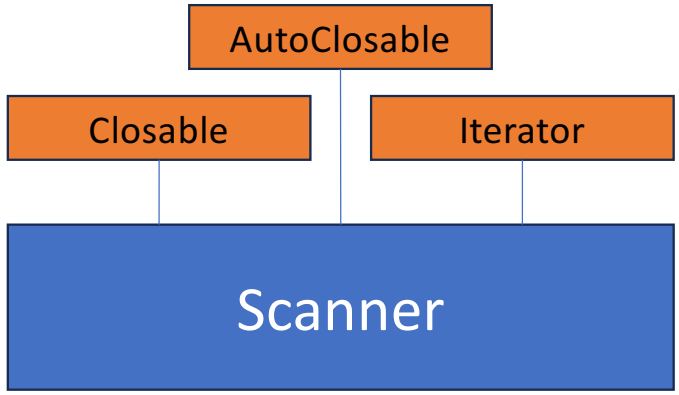
```
public class Root2 {
   public static void main(String[] args) {
      int i = 2;
      double r = Math.sqrt(i);

      System.out.format("The square root of %d is %f.%n", i, r);
   }
}
```

```
public class Format {
   public static void main(String[] args) {
      System.out.format("%f, %1$+020.10f %n", Math.PI);
   }
}
```

THE SCANNER

- Scanner is a class useful to break down formatted input and break input into tokens




```
String input = "1 fish 2 fish red fish blue fish";
Scanner s = new Scanner(input).useDelimiter("\\s*fish\\s*");
System.out.println(s.nextInt());
System.out.println(s.nextInt());
System.out.println(s.next());
System.out.println(s.next());
s.close();
```

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
```

```
Scanner sc = new Scanner(new File("myNumbers"));
while (sc.hasNextLong()) {
    long aLong = sc.nextLong();
}
```

THE SCANNER

- The Scanner has several interesting methods
  - findInLine()
  - findWithinHorizon()
  - hasNext …..
    - hasNextFloat()
    - hasNextInt()
    - hasNextBoolean()
    - and so on…
  - next()
  - match()
- Combined with regex it can do sophisticated pattern matching and data extraction from any string or stream

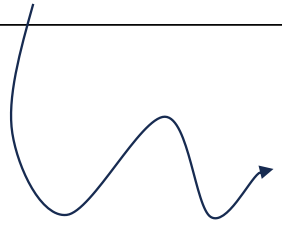
Example using match

```
String input = "1 fish 2 fish red fish blue fish";
Scanner s = new Scanner(input);

s.findInLine("(\\d+) fish (\\d+) fish (\\w+) fish (\\w+) fish");

MatchResult result = s.match();

for (int i=1; i<=result.groupCount(); i++)
  System.out.println(result.group(i));
s.close();
```



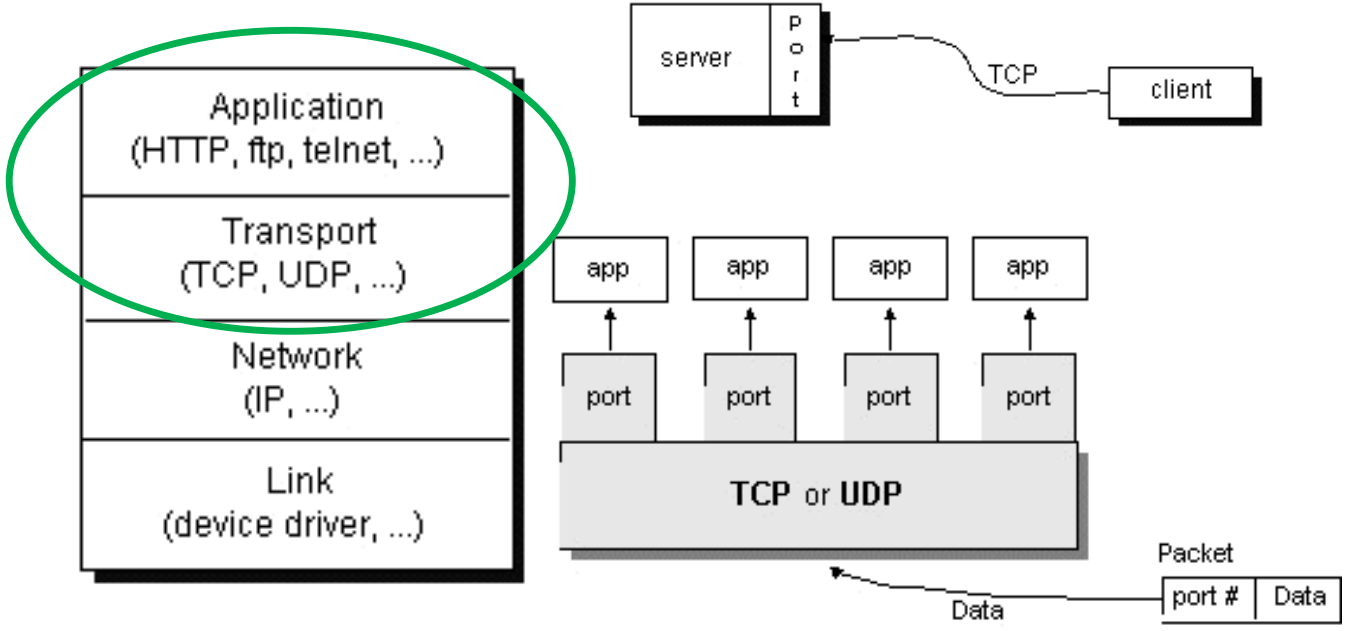
```
Group 1: (\\d+) matches 1
Group 2: (\\d+) matches 2
Group 3: (\\w+) matches red
Group 4: (\\w+) matches blue
```

AGENDA

- Input and outputs
- Data streams
  - Byte streams
  - Character streams
  - Buffered streams
  - Data streams
  - Object serialization
  - I/O from keyboard
- Network I/O
  - UDP sockets
  - TCP sockets
- Events
  - Graphical types of events
  - Network types of events
- The important stuff
- Assignments

QUICK COMMUNICATION OVERVIEW

- Use of standardized protocols allows for (easy) communication
- Most of the stack is already a part of the host operating system
- Java focuses on transport layer and above
- Java's networking classes abstract away everything below and we never write code to handle IP packet routing or TCP handshakes, the OS and Java runtime handle all of that



COMMUNICATION – TCP SOCKETS

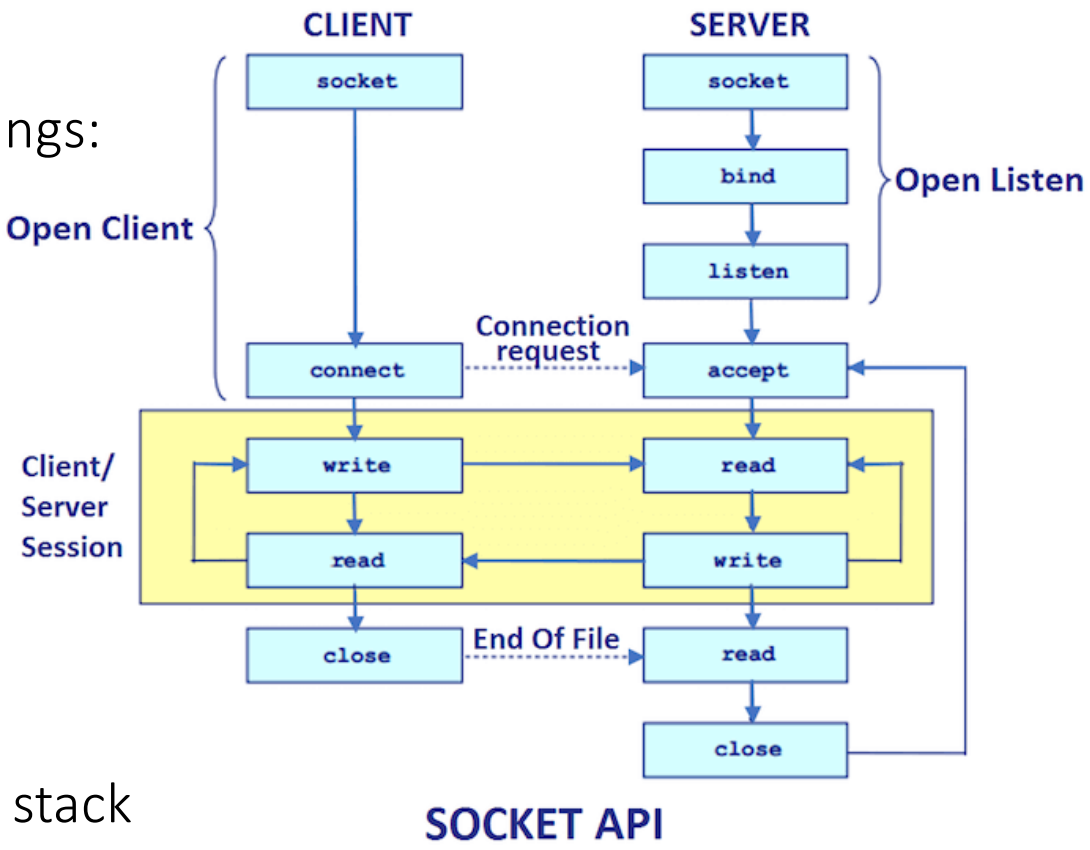
- A socket is essentially a plug point for network communication
- A socket is a combination of three things:
  - An IP address (who to talk to)
  - A port number (what service)
  - A protocol (usually TCP or UDP)
- Server-side sockets

```
ServerSocket ss=new ServerSocket(6666);
Socket s=ss.accept();
```

- Client-side sockets

```
Socket s=new Socket("localhost",6666);
```

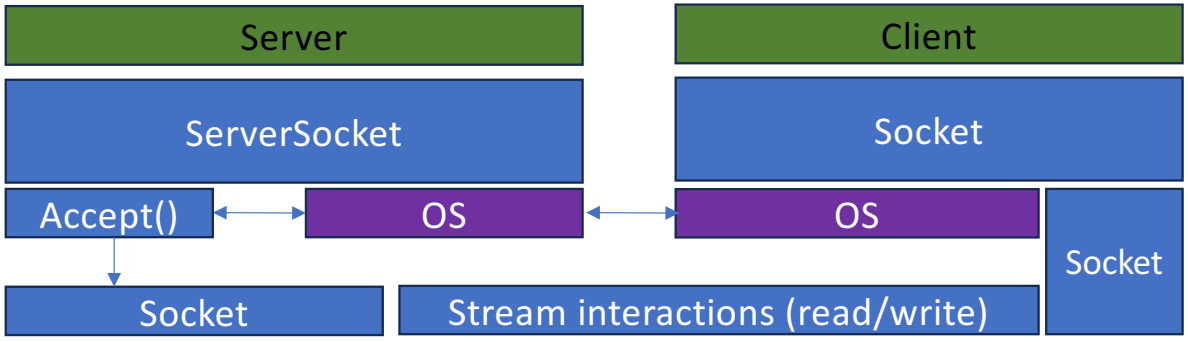
- This type of sockets relies on the TCP stack in the OS

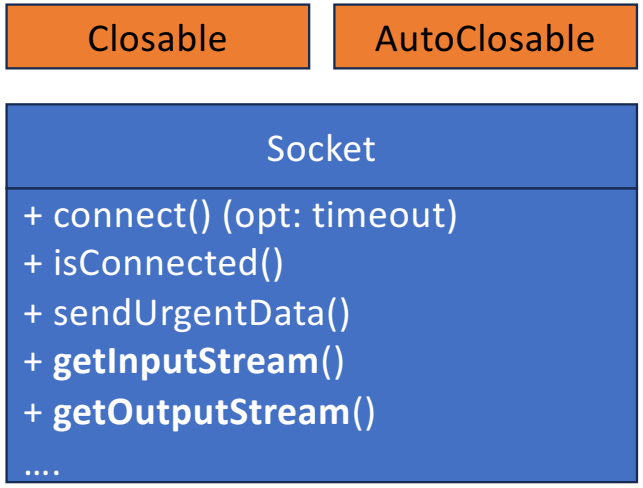


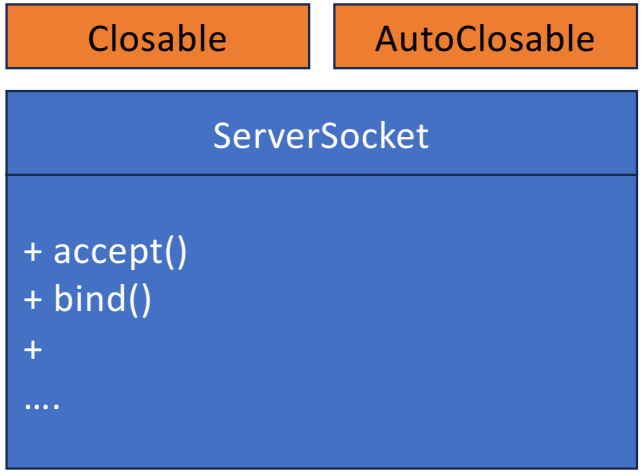
https://www.javatpoint.com/socket-programming

COMMUNICATION – TCP SOCKETS

- ServerSocket ensures that we get a dedicated socket to interact with the client
  - Accept() returns the socket
- The socket provides us with the stream that we can then use to read/write with
  - And read/write goes both ways
  - TCP is connection oriented and a connection must be established before any data flows







COMMUNICATION – UDP SOCKETS

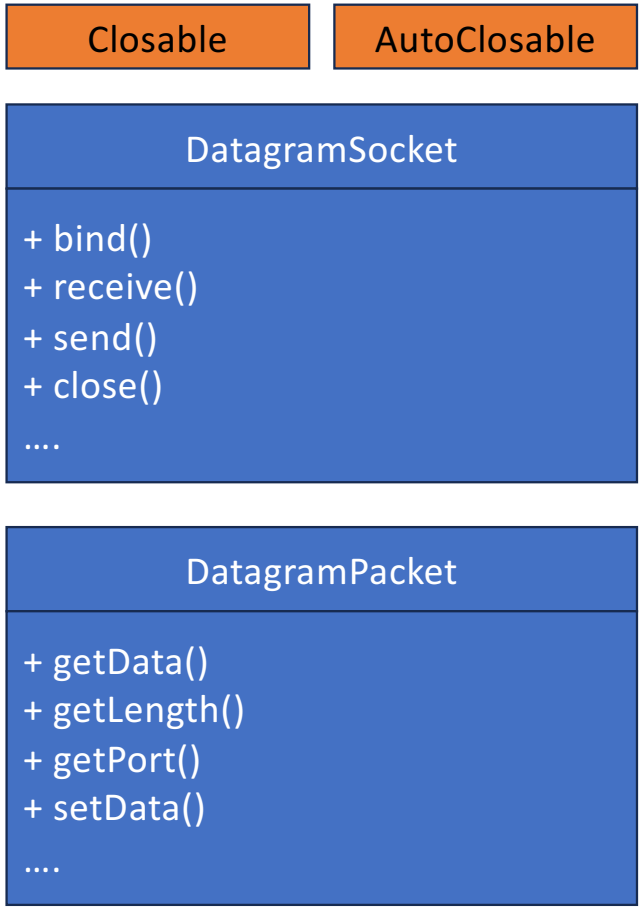
- User Datagram Protocol (UDP) is the postcard
- We just worry about sending and receiving, UDP is that simple
- Two equivalent approaches to setup a UDP socket on port 8888

```
DatagramSocket s = new DatagramSocket(null);
s.bind(new InetSocketAddress(8888));
```

```
DatagramSocket s = new DatagramSocket(8888);
```

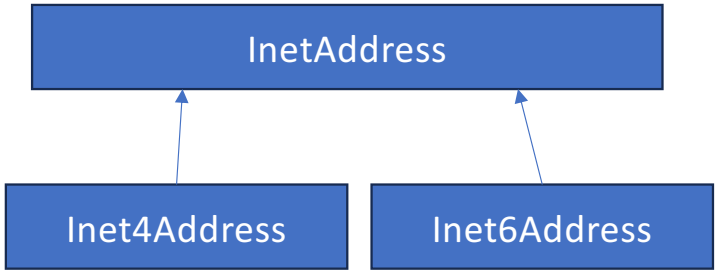
- Receiving from a UDP port is also simple

```
byte[] buffer = new byte[512];
DatagramPacket response = new DatagramPacket(buffer, buffer.length);
socket.receive(response);
```



INETADDRESS

- InetAddress is Java's way of representing a network address
- It wraps either an IP address or a hostname into a single object and tells the package where to go
- InetAddress.getByName() handles both IPv4 and IPv6 automatically



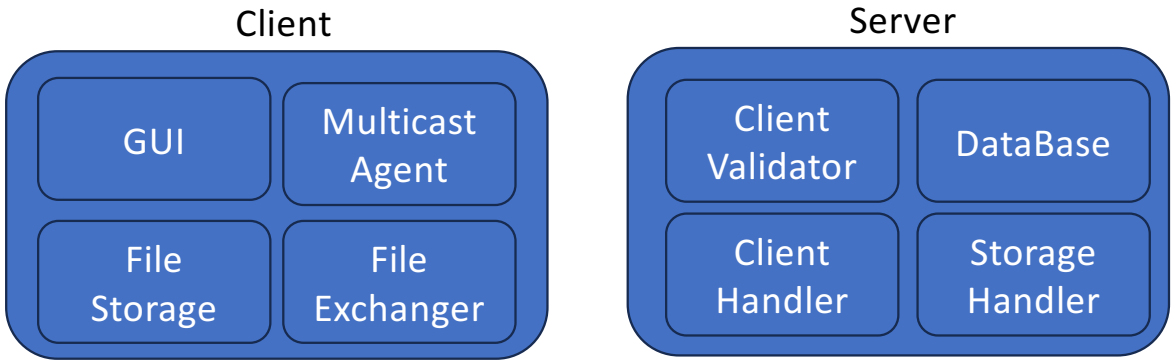

```
InetAddress addr = InetAddress.getByName("127.0.0.1");
```

```
byte[] ipAddr = new byte[]{0x7f,0x00,0x00,0x01};
InetAddress addr = InetAddress.getByAddress(ipAddr);
```

```
InetAddress IPv6 =
InetAddress.getByName("2001:db8:3333:4444:5555:6666:1.2.3.4");
```

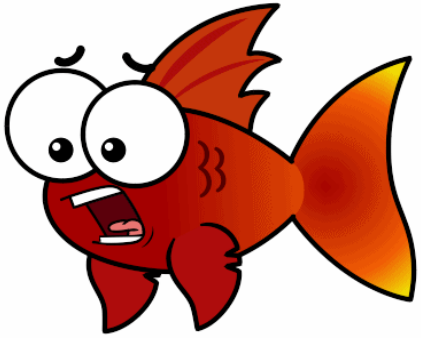
DESIGNING COMMUNICATION ENTITIES

- Model what you have designed
- Often a (sub)module can also be expressed as a class
  - Specialization may happen in client and server part
- With benefit one can extend from existing classes, e.g. socket or serversocket
  - Beware – often it is simpler to just include such as attributes



The end result should be code that:
- aligns very well with your documentation
- is intuitively easy to understand
- is logically structured
- is easy to maintain
- is easy to spot bugs in
- not very long code segments

BREAK – PAUSE(FISK)

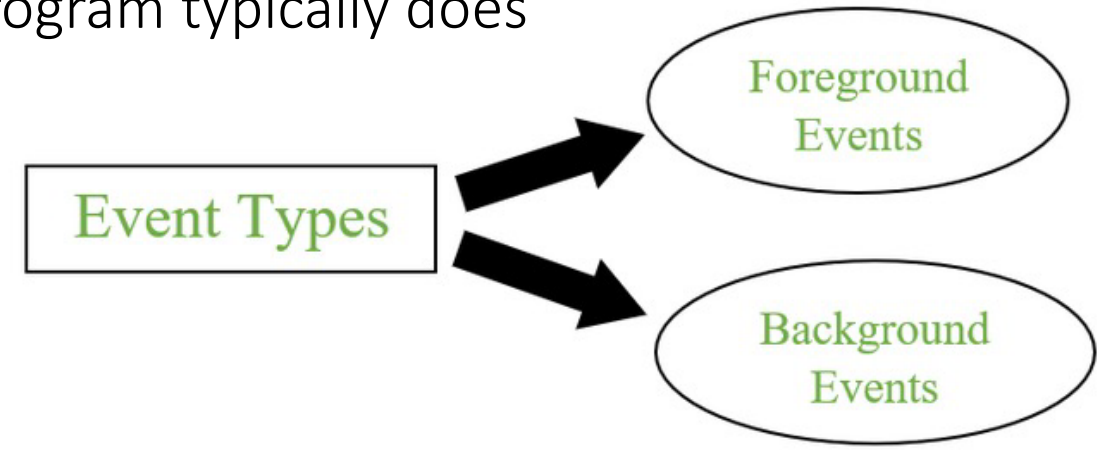


AGENDA

- Input and outputs
- Data streams
  - Byte streams
  - Character streams
  - Buffered streams
  - Data streams
  - Object serialization
  - I/O from keyboard
- Network I/O
  - UDP sockets
  - TCP sockets
- Events
  - Graphical types of events
  - Network types of events
- The important stuff
- Assignments

EVENTS

- *Events* are similar to Exceptions, in that they are generated when "something happens"
- They are typically generated in connection with user interaction, e.g., mouse clicks etc.
- Unlike Exceptions, they are not thrown down the call stack, but rather handed over to an *Event Listener* which acts as an event handler
- If they are not caught, the program typically does not die
- No try-catch construct, just ordinary method calls

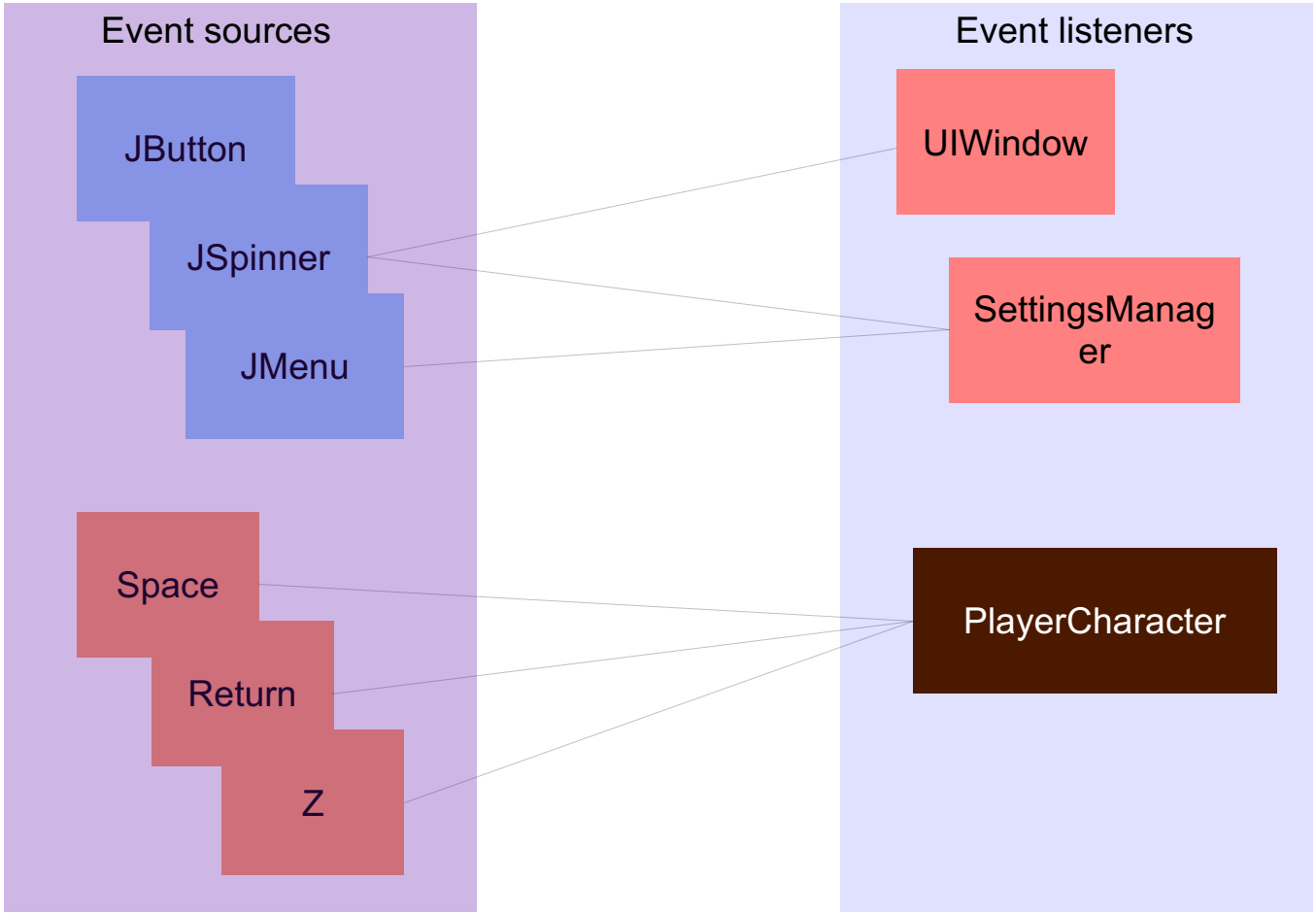


EVENT TERMINOLOGY

- ***event***: A type of thing that can happen
- ***event firing***: A specific occurrence of an event; an event happening
- ***event listener***: Something that looks out for event firings
- ***event handler***: Something that occurs when an event listener detects an event firing
- ***event subscriber***: A response that the event handler is supposed to call

```
Something happens     →    detected by    →    triggers
(event firing)          (event listener)    (event handler)
                                                   |
                                                   ↓
                                              calls subscriber
                                              (actual response)
```

THE EVENT LISTENER MODEL (GUI)



EVENTS IN JAVA (GUI)

- First thing to do is add one or more *event listener* objects to the *event source* object(s) involved

```
public class Simulator extends JFrame implements ActionListener {
 // Constructor
 public Simulator() {
        JButton startBtn = new JButton("Start");
        JButton stopBtn = new JButton("Stop");
        startBtn.addActionListener(this);
        stopBtn.addActionListener(this);
    ...
```

- This way, both startBtn and stopBtn can hand over Action events to the Simulator object

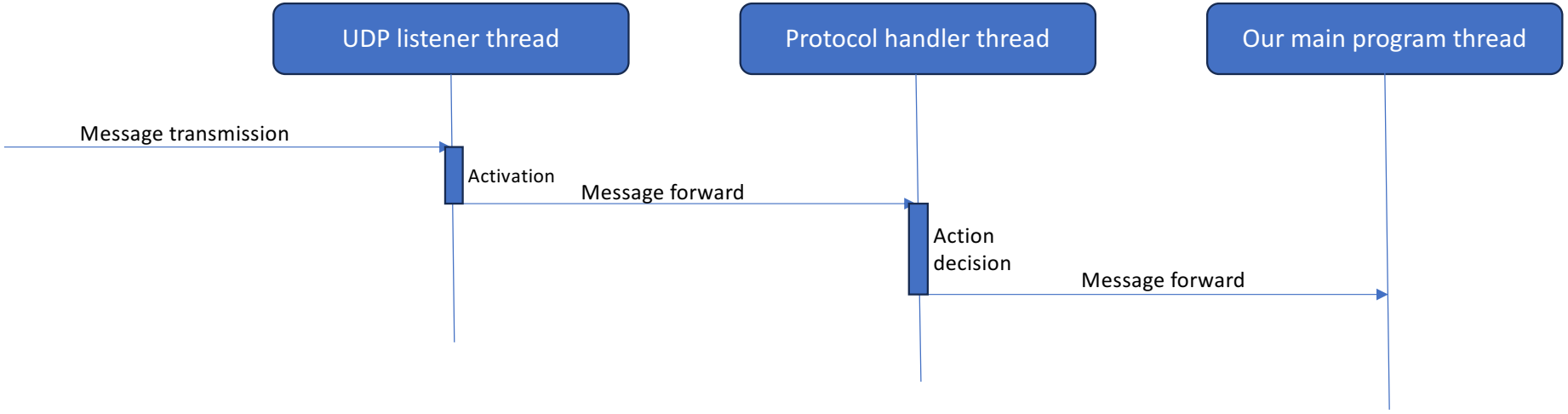
EVENTS IN JAVA (GUI)

- To receive events, an appropriate event handling method must be implemented:

```
public class Simulator extends JFrame implements ActionListener {
 ...
 public void actionPerformed(ActionEvent ae) {
    String buttonLabel = ae.getActionCommand();
    if (buttonLabel.equals("Start")) {
       startSimulation();
    } else {
       stopSimulation();
    }
 }
```

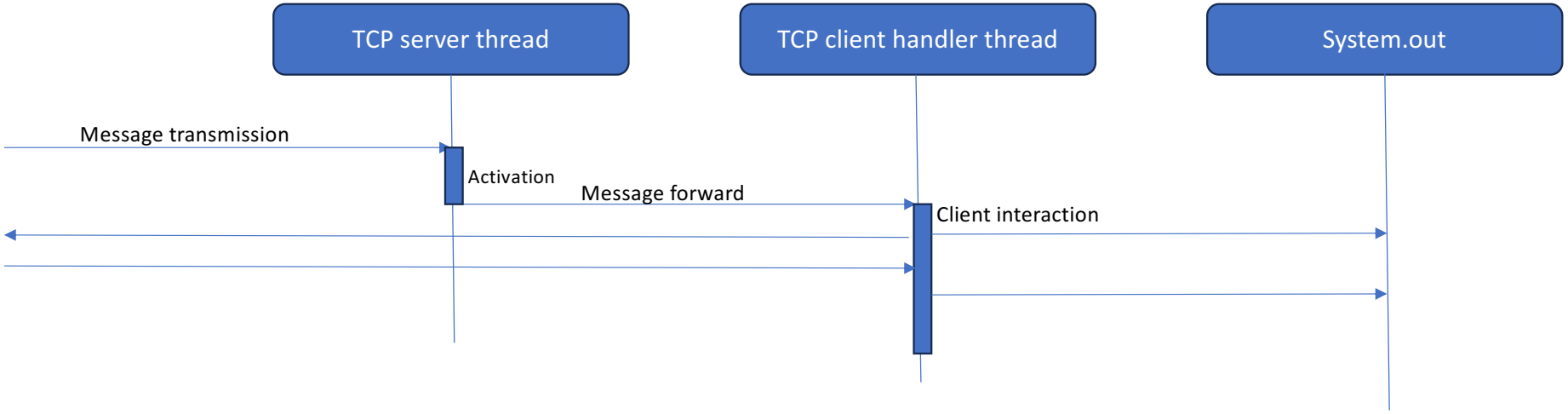
EVENTS WITH NETWORKING (UDP)

- We may wish to create and handle an event upon receiving a UDP packet
- The UDP listener threads waits in blocked mode upon a UDP to be received
- Once received the UDP listener triggers a cascade of events (method calls)



EVENTS WITH NETWORKING (TCP)

- We can also separate threads upon client connections (TCP event)
- Client connects to the server – threading and handling the client separately, allows for more clients to connect
- From then, we can do message exchange or whatever



AGENDA

- Input and outputs
- Data streams
  - Byte streams
  - Character streams
  - Buffered streams
  - Data streams
  - Object serialization
  - I/O from keyboard
- Network I/O
  - UDP sockets
  - TCP sockets
- Events
  - Graphical types of events
  - Network types of events
- The important stuff
- Assignments

THE IMPORTANT STUFF

- Java I/O is built on a layered pattern, and Streams are always wrapped around each other to add functionality
- Choose the right stream for the right job. The naming convention tells you everything — Reader/Writer for characters, and for bytes we use InputStream/OutputStream
- Always close your streams — preferably with try-with-resources
- Sockets abstract network communication into familiar I/O. Once a socket is open, reading and writing over a network looks almost identical to reading and writing a file
- Event driven programming decouples who fires from who responds. This is fundamentally different from direct method calls and is what makes large systems manageable

AGENDA

- Input and outputs
- Data streams
  - Byte streams
  - Character streams
  - Buffered streams
  - Data streams
  - Object serialization
  - I/O from keyboard
- Network I/O
  - UDP sockets
  - TCP sockets
- Events
  - Graphical types of events
  - Network types of events
- The important stuff
- Assignments

ASSIGNMENTS

1) Split your train program with the two types of threads into two separate programs (projects). One is the client program which contains the user input, and one is the server program which handles the list of trains
2) Extend the train list server to also from time to time write the list to a file. If we wish to restart the server part, we should also read the file and initiate the list with the saved list
3) Extend the client/server to allow the interaction to happen across a network. Use your local network and you fellow students' laptop to run. You may need to open a port on the firewalls of the machines involved. Feel free to use a virtual machine or two instead as an alternative or just use the internal loopback interface (127.0.0.1) and start the client/server as individual programs.
   a) First use the clients with manual inputs/edits
   b) Then extend the client design and implementation to trigger a random action based upon a mouse click
      i. How fast can you "click" a request before you run into some issues? What type of issues to you experience? Can you do something about this?
   c) Then use the clients with automatic actions i.e. the clients should randomly without user interaction select a query, send it to the server and show the response
      i. How fast can you have a large (as many as feasible) number of clients execute random queries, as fast as possible, without running into problems? What kind of problems do you observe? What can you do about these problems?
      ii. Does it matter if you use a TCP or UDP socket for the communication?